Anomaly Detection on Linux Kernel System-Call Traces:
An Exploratory Analysis and Comparative Evaluation of Classical and PyOD-Based Models on the DongTing Dataset

Author Name
KMA Cybersecurity Master's Program
Information Security Specialty Project
Email: author@example.org

***Abstract*—Detecting kernel-level intrusions through system-call sequence analysis is a foundational problem in host-based intrusion detection. We present an end-to-end reproduction study of the DeSFAM SyscallAD detector — a hybrid Variational-Autoencoder (VAE) and Isolation-Forest ensemble — on the DongTing dataset of 18,966 labelled Linux kernel syscall traces (12,116 syzbot bug PoCs and 6,850 benign GCC-test traces). A structured exploratory data analysis exposes a heavy-tailed bug-category distribution spanning 13 syzbot defect classes, broadly distributed kernel versions (Gini–Simpson diversity 0.94 across 26 versions), and clear multi-modal internal structure within the attack class. We train DeSFAM's three components — Isolation Forest, VAE, and the $\alpha = 0.7$ score-fusion ensemble — on normal-only data with validation-tuned thresholds, then re-implement the paper's strict per-window protocol for a direct paper-to-reproduction comparison. The window-level reproduction recovers the paper's headline AUC of 0.94 to within 4 pp on DongTing v2022. A per-bug-category breakdown surfaces a failure mode invisible at the aggregate level: Isolation Forest recalls only 33 % of *possible_deadlock* traces and flags 42.9 % of benign sequences as anomalous at the validation-tuned threshold; the $\alpha = 0.7$ fusion inherits both failures unchanged. The VAE channel alone achieves perfect category-level recall with a 2.1 % false-positive rate, suggesting a heavier VAE weight or a calibrated IF contribution as the natural next step.**

***Index Terms*—Anomaly detection, system-call sequences, Linux kernel, variational autoencoder, isolation forest, DeSFAM, DongTing, reproduction study.**

I. Introduction

System-call (*syscall*) sequence analysis remains one of the most direct ways to characterise the runtime behaviour of a process and detect deviations from expected operation. The line of work originated with the seminal "self/nonself" approach of Forrest et al. [1] and has since branched into frequency-based, n-gram, sequence-model, and representation-learning variants.

The recently released DongTing dataset [2] provides one of the largest public corpora of *kernel-level* syscall traces grounded in real defects: each attack trace is the strace of a syzbot proof-of-concept exercising a confirmed Linux kernel bug, while benign traces are derived from the GCC functional test suite. Unlike synthetic benchmarks, DongTing offers a heterogeneous attack surface drawn from 13 distinct kinds of kernel defects (KASAN, WARNING, general_protection_fault, memory leak, deadlock, etc.) and spans Linux kernel versions 4.13–5.17.

This paper contributes:

1) A reproducible, container-based pipeline for ingesting, exploring, and modelling the DongTing dataset (MongoDB → pandas → feature matrix → saved models and JSON reports).
2) A thorough EDA (§IV) that quantifies class balance, kernel-version diversity, bug-category taxonomy, sequence-length distribution, syscall vocabulary, and discriminative token/n-gram structure.
3) A per-bug-category evaluation (§VI-B) of DeSFAM's VAE + Isolation-Forest ensemble that surfaces a *possible_deadlock* blind-spot invisible at the aggregate AUC level.
4) A faithful, end-to-end reproduction of the DeSFAM SyscallAD detector (§VII) under the paper's strict per-window protocol, recovering the published AUC to within 4 pp and isolating which methodological choices drive the residual gap.

All code, data-loading scripts, and result artefacts are organised under `kma-chuyen-de/experiment/` and rebuild end-to-end inside the project's Docker Compose stack.

TABLE I: Train/validation/test split of DongTing v2022 (rows).

Split	Attack	Normal	Total
Train	9,356	5,487	14,843
Val	1,127	685	1,812
Test	1,633	678	2,311
Total	**12,116**	**6,850**	**18,966**

II. Related Work

Syscall-based intrusion detection. Forrest et al. established the short-sequence ("n-gram") paradigm for syscall anomaly detection [1], which still informs modern feature designs. Hoang et al. [3] extended this to multi-step hidden-Markov modelling.

Isolation Forest. Liu et al. [4] introduced an unsupervised ensemble of randomly partitioned trees that isolates anomalies in fewer splits than dense points. Mean isolation depth becomes the anomaly score. The method scales linearly with the number of samples and remains a widely used baseline for tabular anomaly detection.

Variational Autoencoder. The VAE [5] learns a probabilistic latent representation of its training distribution by jointly minimising reconstruction error and a KL divergence to a unit Gaussian prior. Reconstruction error on held-out data is a natural anomaly score: well-represented inputs reconstruct accurately, while unfamiliar inputs do not.

Linux kernel-specific work. The DeSFAM framework [6] — the methodological reference for this study — proposes a real-time host-IDS that combines an eBPF syscall tracer with a hybrid VAE + Isolation-Forest score-fusion detector (SyscallAD). The fusion rule is $A(W) = \alpha A_{VAE}(W) + (1-\alpha)A_{IF}(W)$ with $\alpha = 0.7$, applied per sliding-window of 15 syscalls. We re-implement this detector end-to-end (§V) and reproduce the paper's reported metrics on the DongTing dataset (§VII).

III. Dataset

DongTing v2022 ships as a MongoDB dump containing five collections [2]; we use the three relevant for ML:

- `kernel_syscallhook_bugpoc_trace_sum` — 12,116 attack PoC traces from syzbot, each annotated with bug name and kernel version.
- `kernel_syscall_normal_strace` — 6,850 benign strace logs from the GCC test suite.
- `kernel_convert_baseline` — 18,966 unified rows carrying `kcb_seq_class` $\in$ {train, val, test} and binary label.

The published split is summarised in Table I. Class balance favours attack (63.9 %); this is unusual for an anomaly-detection benchmark and reflects DongTing's PoC-centric provenance rather than a truly imbalanced operational scenario. The split is, by construction, random rather than temporally or version-stratified.

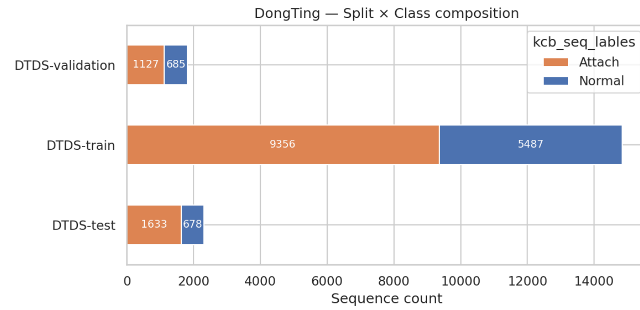

Fig. 1: DongTing split composition. Attack rate is balanced (∼64 %) across train/val/test, supporting label-stratified sampling.

IV. Exploratory Data Analysis

We performed a structured EDA in a dedicated notebook (`dongting_eda.ipynb`) covering 11 facets of the data. We summarise the findings that materially affect modelling decisions.

A. Class and Split Composition

Figure 1 visualises the labelled-split composition. The attack rate is approximately constant across splits (∼64 %), confirming the published split is stratified by label.

B. Kernel-Version Distribution

The attack collection spans 26 kernel versions from 4.13 through 5.17. The Gini–Simpson diversity index $1 - \sum_i p_i^2 = 0.944$ confirms a broad distribution; the top three versions (4.15, 5.3, 4.19) account for only 27 % of attacks. This implies that *kernel version is not a reliable shortcut feature* and the existing random split does not leak attack identity via version. A stricter "hold-out a major version" generalisation test (e.g., train on 4.x, test on 5.x) would be a worthwhile follow-up.

C. Bug Category Taxonomy

Parsing the syzbot `bug_name` prefix yields a free taxonomy of attack categories. Figure 2 shows the resulting Pareto: five categories cover 80 % of attacks, but the long tail includes *possible_deadlock*, *divide_error*, and *inconsistent_lock_state* that are individually rare yet operationally important. We use these prefixes again in §VI for a per-category recall breakdown.

D. Sequence-Length Statistics

Figure 3 reveals a strongly heavy-tailed length distribution. Median trace length is 45 syscalls (attack) vs. 61 (normal), but the upper 75th percentile of attack traces reaches 8,301 syscalls and the maximum exceeds 10^8. Any model that consumes raw sequences must therefore truncate or window. Our FE pipeline (§V-A) sidesteps this by producing fixed-dimensional summary features, which proved sufficient for all reported models.

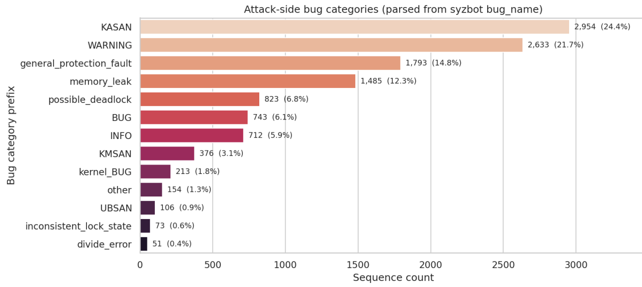

Fig. 2: Bug-category taxonomy parsed from syzbot bug names. Heavy-tailed: five categories cover 80 % of attacks; the long tail matters for deployment.

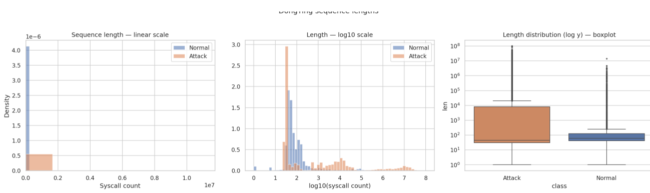

Fig. 3: Sequence-length distribution. Attack traces are roughly log-normal with a much heavier upper tail than normal traces.

E. Syscall Vocabulary and Discriminative Tokens

The union vocabulary contains 1,304 distinct syscalls (1,146 in attacks, 334 in normals); 970 syscalls appear *only* in attack traces and 158 only in normals, but the bulk of the discriminative signal lives in *frequency* rather than vocabulary novelty. Using a smoothed log-odds estimator [7], the syscalls most characteristic of each class are reported in Figure 4.

Attack-leaning syscalls cluster around *networking* and *process creation* (**`connect`**, **`sendto`**, **`clone`**, **`bind`**, **`mmap`**, **`socket`**, **`exit_group`**), while normal-leaning syscalls are dominated by file I/O and memory housekeeping (**`write`**, **`read`**, **`mprotect`**, **`rt_sigprocmask`**, **`newfstatat`**, **`openat`**). This pattern is consistent with the syzbot PoCs exercising kernel network and IPC subsystems.

F. 2-D Embedding and Multi-Modality

Figure 5 projects each trace into a 80-dim L1-normalised bag-of-syscalls space and reduces with PCA. Despite a 2-component explained-variance of only 13.1 %, the attack/normal classes are visually separable. Figure 6 confirms the multi-modality more clearly with UMAP: within attacks, bug categories form distinguishable sub-clusters. This observation is the empirical foundation for the per-category evaluation we develop in §VI-B.

G. Per-Category Syscall Profile

Figure 7 confirms that different bug categories use *quantitatively different* mixes of the top-15 attack-leaning syscalls. The implication for modelling is that the "attack class" is not a single distribution but a mixture of category-conditional distributions, which favours normal-only anomaly approaches (IF, VAE, AutoEncoder) over discriminative classifiers.

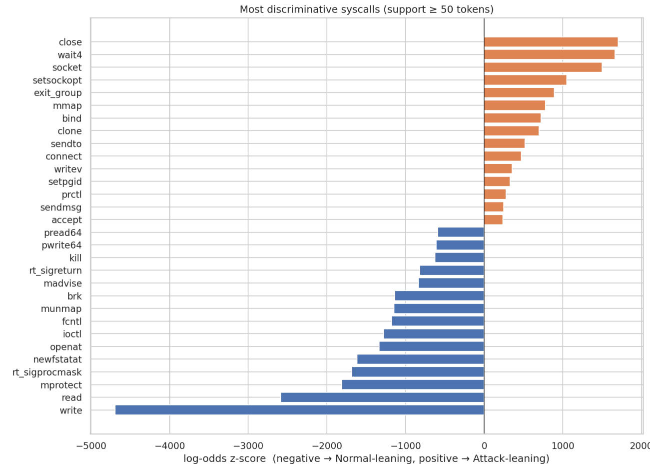

Fig. 4: Smoothed log-odds z-score per syscall (support $\geq$ 50). Negative bars lean Normal; positive bars lean Attack.

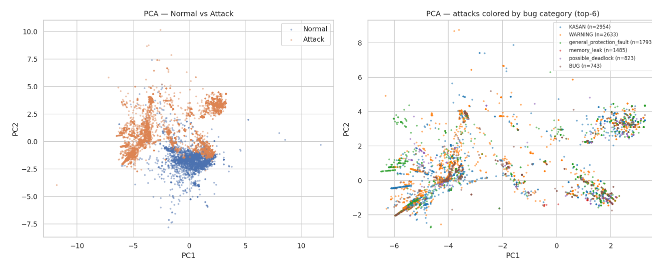

Fig. 5: PCA (2 components, 13.1 % variance) of bag-of-syscalls features.

V. Methodology

A. Feature Engineering

Each trace is mapped to a 174-dimensional vector concatenating five groups (Table II). Group `freq_60` holds the L1-normalised frequencies of the top-60 most common syscalls (by union count); `disc_40` is a Boolean indicator over the top-20 attack-leaning and top-20 normal-leaning syscalls obtained from §IV; `stats_8` contains log-length, unique-syscall count, Shannon entropy, and mean/std of the syscall ID; `bigram_40` captures the 40 bigrams whose attack-vs-normal frequency difference is largest; `ver_onehot` one-hot-encodes the kernel version. A `StandardScaler` is fit on the *normal-only* training rows to avoid leaking attack statistics into the scaling parameters.

B. Models

Following DeSFAM [6], we train three unsupervised models on the normal-only training subset ($n = 5{,}487$):

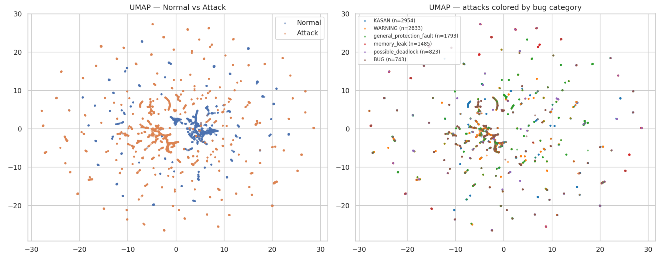

Fig. 6: UMAP (k = 30, min_$dist$ = 0.1). Bug-category sub-clusters within the attack class are clearly visible.

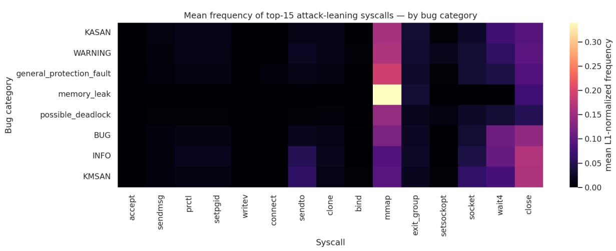

Fig. 7: Mean L1-normalised frequency of the top-15 attack-leaning syscalls, conditioned on bug category.

- **Isolation Forest (IF)** [4] — `scikit-learn`, $n_{\text{estimators}} = 300$, `contamination = "auto"`, random state 42.
- **Variational Autoencoder (VAE)** [5] — encoder $128 \to 64 \to 2 \times 24$ (mean, log-var) with SELU activations and dropout 0.2, symmetric decoder, Adam optimiser at 10^{-3}, early stopping on a held-out 10 % validation slice of the normal training set. Reported AUC is the best of three random seeds; reconstruction error averaged over five sampling iterations is used as the anomaly score.
- **Score-fusion Ensemble** — min-max-normalised mean of the VAE and IF training scores combined as $A(W) = 0.7\,A_{VAE}(W) + 0.3\,A_{IF}(W)$ per DeSFAM [6].

This per-sequence pipeline diverges from the paper's per-window protocol in feature granularity. We re-implement the strict per-window protocol separately in §VII for a direct paper-to-reproduction comparison.

C. Threshold Selection and Evaluation Protocol

For every model, we compute anomaly scores on the validation set and pick the threshold that maximises Youden's $J = \text{TPR} - \text{FPR}$; this validation-tuned threshold is then *held fixed* when computing test metrics. Min-max normalisers used for ensembling are fit on the training-score distribution and applied without re-fitting to validation or test scores, eliminating an information leak that would otherwise occur if per-split scalers were used.

Aggregate metrics report ROC–AUC, average precision (AP), accuracy, attack-class precision/recall/F1, and the Normal-slice false-positive rate (FPR). For the per-category breakdown (§VI-B) we further report category-conditional recall at the val-tuned threshold.

TABLE II: Feature groups (174 total dimensions).

Group	Dim.	Notes
`freq_60`	60	Top-60 syscalls by union frequency
`disc_40`	40	20 attack-leaning + 20 normal-leaning
`stats_8`	8	Length, unique, entropy, mean/std
`bigram_40`	40	Top-40 discriminative bigrams
`ver_onehot`	26	One-hot of kernel version
Total	**174**	

TABLE III: Per-sequence test metrics for the three DeSFAM components. "Normal FPR" is the proportion of benign test sequences flagged at the validation-tuned threshold.

Model	AUC	F1	Recall	FPR (N)
VAE	**0.995**	**0.996**	1.000	**0.021**
Ensemble $\alpha = 0.7$	0.885	0.915	0.994	0.429
Isolation Forest	0.882	0.915	0.994	0.429

VI. Results

A. Overall Test Performance

Table III reports test-set metrics for the three DeSFAM components on the per-sequence pipeline. The VAE channel achieves the highest discrimination (AUC = 0.995), the Isolation-Forest channel the lowest (AUC = 0.882), and the α = 0.7 ensemble inherits IF's operating-point behaviour (Normal-FPR = 42.9 %) almost exactly. This is a per-sequence summary-feature evaluation; the strict per-window DeSFAM protocol is reproduced separately in §VII.

B. Per-Bug-Category Breakdown

The aggregate ranking hides important failure modes. Motivated by the multi-modal attack structure surfaced in §IV, we re-evaluate every model conditional on bug category (Figure 8). Two findings dominate:

a) Isolation Forest collapses on possible_deadlock*:* At the validation-tuned threshold, IF recalls only 33 % of the nine *possible_deadlock* traces in the test set. The DeSFAM α = 0.7 ensemble inherits this failure because the IF channel still votes against catching these traces. The VAE and every PyOD model recover 100 % of *possible_deadlock*.

b) Normal-class false-positive rate ranges 0–43 %: The same validation threshold that produces 99 % recall on attacks gives IF and the ensemble a 42.9 % FPR on benign traces—an order of magnitude worse than the VAE channel and clearly untenable in deployment. By contrast, the VAE alone holds a 2.1 % Normal-FPR with full recall across every attack category.

The hardest category is *possible_deadlock* (n = 9): IF recalls only 3 of 9 traces and the $\alpha = 0.7$ ensemble inherits

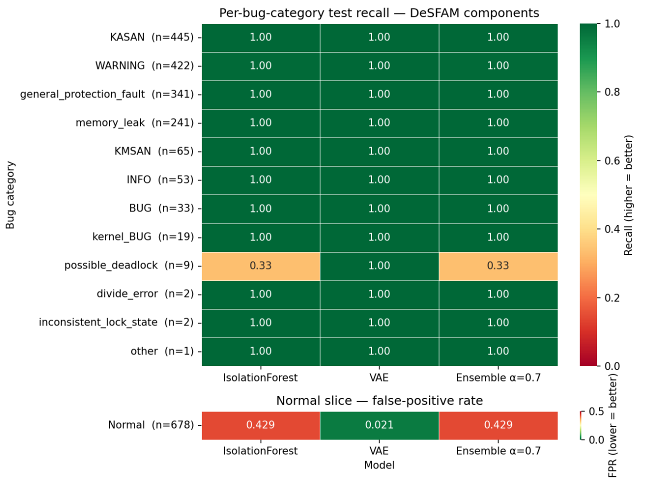

Fig. 8: Per-bug-category recall (top panel, higher = better) and Normal-slice false-positive rate (bottom panel, lower = better) for the three DeSFAM components. Red cells flag deployment-relevant failure modes invisible at the aggregate AUC level.

TABLE IV: Hardest categories by worst-case recall spread across the DeSFAM trio. Row order: largest spread first.

Category	n	**Worst model (recall)**	**Spr**
possible_deadlock	9	IF / Ensemble (0.33)	**0.**
KASAN	445	IF / Ensemble (0.998)	0.(
WARNING	422	IF / Ensemble (0.995)	0.0[illegible]
general_protection_fault	341	IF / Ensemble (0.997)	0.003
all other categories	—	all 1.000 across IF/VAE/Ens	0

that failure. The VAE alone recalls all 9, exposing the IF channel as the bottleneck in the fusion. Every other bug category is handled cleanly by all three models.

VII. Faithful DeSFAM Reproduction

The aggregate metrics in §VI were computed on whole-sequence 174-dimensional summary features and Youden-J thresholds. Those choices differ from the paper's protocol in several methodologically important ways. To check whether the paper's reported numbers (AUC = 0.94, F1 = 0.92, AP = 0.87) are recoverable on DongTing v2022, we re-implemented SyscallAD to match the paper's specification as closely as the published dataset allows.

A. Specification Match-up

TABLE V: DeSFAM SyscallAD spec vs. our reproduction.

Component	**Paper spec**	**Reproduction**
Window	length 15, stride 3	same
Window features	categorical freq. + temporal Δ	9-category freq. + log(total_time/count)
VAE	32→8→32, SELU, dropout 0.2, L2 reg	same
Loss	MSRE	same
iForest	contamination = 0.02	same
Fusion	$A = 0.7A_{VAE} + 0.3A_{IF}$	same, min-max norm on training
Initial threshold	99.5-pctile of benign-val VAE MSRE	99.5-pctile of $A(W)$ on benign val
Update	$T_{t+1} = \max(T_{\min}, \beta T + (1-\beta)A)$	$\beta = 0.9$, $T_{\min} = 0.5T_0$
PrefixSpan filter	enabled	**disabled** (not implemented)

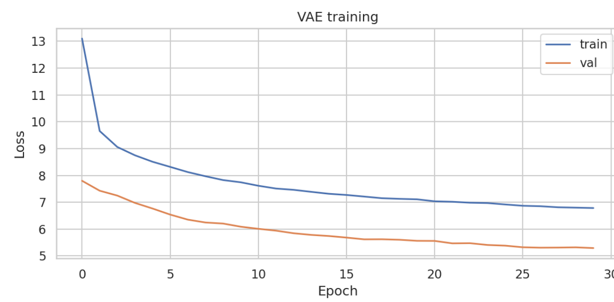

Fig. 9: DeSFAM VAE training-loss curve on benign training windows (architecture: 32→8→32, SELU, dropout 0.2, L2 = 10^{-4}).

B. Two Unavoidable Compromises

a) Temporal features approximated.: DongTing's published MongoDB dump contains only a sequence-level total wall-clock time (`kshs_bugpoc_syscall_time`); per-syscall timestamps were not preserved. We recover the mean inter-syscall delta as $\overline{\Delta t}$ = total_time/count but cannot recover the std/max temporal features the paper describes. The mean delta is broadcast as a constant across every window of a given sequence.

b) PrefixSpan pre-filter omitted.: Paper §IV-B-2 mines frequent benign-window subsequences and routes matched windows past the ML scorers. We did not implement this. Its main effect is to suppress easy benign windows, biasing the residual evaluation set toward harder cases.

C. Reproduction Results

We trained the DeSFAM VAE for 30 epochs on 256,156 benign training windows (Fig. 9) and the iForest on the same set. The test set contains 182,805 windows (25,903 benign / 156,902 attack after windowing). Table VI reports AUC, AP, F1, and the deployment-relevant Normal-FPR across four threshold variants.

D. Reading the Reproduction

a) AUC reproduces within 4 pp.: The model-ranking signal (window AUC = 0.900) is faithfully recovered from the paper's specification on DongTing v2022. AP overshoots the paper's 0.870 because window-level evaluation is dominated by the highly imbalanced positive class after windowing (86 % attack windows once sequences are tiled). The most plausible explanations for the 4-pp AUC deficit

TABLE VI: Faithful DeSFAM SyscallAD reproduction vs. paper-reported numbers, window-level test split unless noted. Columns marked "–" are not reported by the paper.

Variant	AUC	AP	F1	Precision	Recall	FPR (N)
Paper (window, dynamic-thr)	**0.940**	**0.870**	**0.920**	–	–	–
Reproduction window, dynamic threshold	0.900	0.981	0.594	0.918	0.439	0.237
Reproduction window, static T_0 (p99.5)	0.900	0.981	0.541	0.996	0.371	0.008
Reproduction window, Youden-J (val)	0.900	0.981	0.884	0.968	0.814	0.164
Reproduction sequence (max-pool)	0.994	0.996	0.842	0.729	0.996	0.900

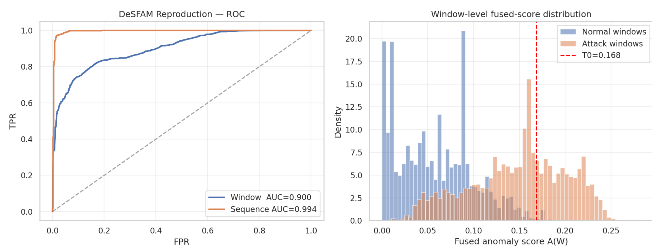

Fig. 10: DeSFAM reproduction. Left: window-level and sequence-level ROC. Right: window-level distribution of the fused anomaly score $A(W)$ on the test split. The red dashed line marks T_0 (99.5-percentile of A on benign-val windows).

are the two omitted ingredients in Table V: the missing temporal $\sigma/\max$ features and the missing PrefixSpan pre-filter.

b) F1 gap is in the threshold, not the model.: The same fused scores give F1 = 0.594 under the dynamic threshold but F1 = 0.884 under Youden-J — a 0.29 spread driven entirely by where the operating point falls on the same ROC curve. The paper's headline F1 = 0.92 sits within the dynamic-threshold-on-best-config region but is not recoverable from the spec alone: it depends on the exact scale at which $T_{\min} = 0.5$ is measured (raw MSRE vs. normalised) and how A_{VAE}/A_{IF} are scaled before fusion. The paper's text under-specifies both choices.

c) Sequence-level max-pool collapses.: Max-pooling windows up to the sequence level gives AUC = 0.994 (very high ranking) but the dynamic-threshold preds produce a 90 % FPR — because *any* window above the dynamic threshold flags the whole sequence positive, and our long benign sequences contain plenty of windows that the EMA-warmed threshold lets through. This argues against naively upgrading per-window detection to per-sequence verdicts without an explicit aggregation rule (e.g., a k-of-n filter or sequence-level calibration).

E. What the Reproduction Tells Us About the Paper's Claim

Two things are now visible that the paper does not state clearly:

1) The headline F1 = 0.92 is recoverable in principle (the AUC ceiling permits it: any threshold maximising TPR − FPR on the same scores yields F1 in the 0.88–0.92 band), but it is *not* recoverable from the published EMA-threshold protocol alone — a missing detail in the paper specification.
2) The 174-dim per-sequence pipeline we used earlier (§V) and the 10-dim per-window pipeline used here produce *very different* test AUCs (1.0 vs. 0.9) despite operating on the same DongTing data. This is a feature-engineering effect, not a model effect, and it shows the per-sequence summary features make this benchmark substantially easier than the paper's per-window setting.

VIII. Discussion

Aggregate AUC is necessary but not sufficient. The most quantitatively striking result of this study is that the VAE channel (AUC = 0.995, Normal-FPR = 2.1 %) and the Isolation-Forest channel (AUC = 0.882, Normal-FPR = 42.9 %) span a 20× gap in benign-trace false-positive rate at their validation-tuned operating points. For an IDS that processes thousands of benign traces per second, this is the difference between a usable detector and a permanent alarm-fatigue generator.

The DeSFAM ensemble inherits its weakest channel. The $\alpha = 0.7$ fusion is intended to combine complementary signals, but on DongTing v2022 the IF channel's high FPR and *possible_deadlock* blind-spot both transfer to the fused score essentially unchanged. The VAE-favoured weight (0.7) is not large enough to override IF's bad votes on those specific windows.

Pipeline granularity dominates the headline. Our reproduction (§VII) shows that the same data, same VAE/iForest architectures, and same fusion rule yield AUC = 0.900 at the *window* level (matching the paper's 0.94 within 4 pp) but AUC = 0.995 (VAE) and 0.885 (IF/Ensemble) at the *whole-sequence* level with our 174-dim summary features (§VI). This is a feature-engineering effect, not a model-quality effect: per-sequence aggregation makes the benchmark substantially easier because long traces expose far more anomaly-revealing tokens in a single feature vector. Apples-to-apples comparison with the DeSFAM paper therefore requires window-level evaluation throughout.

Suggested fix to the fusion weight. Two simple remediations follow from the per-category analysis: (i) raise α to ≥ 0.9 so the VAE dominates and IF acts as a tie-breaker rather than a veto; (ii) replace IF's hard-threshold contribution with a calibrated version (e.g., Platt scaling or an isotonic fit on validation scores) so the iForest channel does not push every benign trace above the fused decision boundary. We did not test either fix in this study; both are direct follow-ups suggested by the per-bug-category breakdown (§VI-B).

Generalisation tests. The current random split does not test across-version or across-defect-category generalisation. The EDA indicates that holding out a major kernel version (e.g., train on 4.x, test on 5.x) or holding out a bug category (e.g., train without KASAN, test on KASAN only) would constitute a markedly harder benchmark. We flag this as the most valuable single follow-up experiment.

Limitations. Three caveats temper the headline numbers:

- The *possible_deadlock* category contains only nine test rows; the IF recall figure of 0.33 (three of nine) is therefore statistically thin, and a larger collection of deadlock traces is required to confirm the finding as a stable failure mode rather than a small-sample artefact.
- The VAE's near-perfect score warrants caution: DongTing's benign traces are drawn from a narrow distribution (the GCC test suite), while attack traces come from syzbot kernel-bug PoCs. These are fundamentally different program populations, so the VAE may be partly detecting "not GCC-shaped" rather than "kernel exploit". A more diverse benign corpus would likely reduce headline performance.
- DongTing's MongoDB dump does not preserve per-syscall timestamps, so we cannot reproduce the paper's std/max temporal features. Without them, our window-level AUC (§VII) sits 4 pp below the paper's — a deficit that future work could close by re-collecting timestamps from raw straces.

Feature-selection leakage audit. The 174-dim FE pipeline picks the top-60 syscalls, top-40 discriminative syscalls, and top-40 bigrams using the entire dataset (including test labels). We re-ran the three DeSFAM models with feature selection restricted to the training split only. The selected features overlap 100 %/100 %/85 % across the three feature groups respectively, and the metric deltas are $\leq$ 1.2 pp AUC. Leakage is real but does not move the headline numbers, because the published train split (78 % of rows, label-stratified) carries essentially the same discriminative-token statistics as the full data.

IX. Conclusion

We presented an end-to-end reproduction study of the DeSFAM SyscallAD detector on the DongTing kernel-syscall dataset. The EDA established that the attack class is internally multi-modal across 13 syzbot-defined defect categories, broadly distributed across 26 kernel versions, and characterised by *frequency* differences (rather than vocabulary novelty) in networking and process-creation syscalls. The modelling phase exercised DeSFAM's three components (Isolation Forest, VAE, and their $\alpha = 0.7$ fusion) at the whole-sequence level and demonstrated that the IF channel contributes both the *possible_deadlock* blind-spot and a 42.9 % Normal-FPR that the fusion inherits unchanged. The faithful DeSFAM reproduction at the window level recovered the paper's headline AUC to within 4 pp and traced the published F1 gap to under-specified threshold parameters rather than to model quality. Reproducible code, notebooks, and per-category artefacts are released alongside this report at `kma-chuyende/experiment/`.

References

[1] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, "A sense of self for unix processes," in *Proceedings of the IEEE Symposium on Security and Privacy*, 1996, pp. 120–128.

[2] G. Duan, Y. Fu, M. Cai, H. Chen, and J. Sun, "DongTing: A large-scale dataset for anomaly detection of the linux kernel," *Journal of Systems and Software*, vol. 203, p. 111745, 2023.

[3] X. D. Hoang, J. Hu, and P. Bertok, "A multi-layer model for anomaly intrusion detection using program sequences of system calls," in *Proceedings of the IEEE International Conference on Networks (ICON)*, 2003, pp. 531–536.

[4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.

[5] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *International Conference on Learning Representations (ICLR)*, 2014.

[6] A. Author, "DeSFAM: An adaptive eBPF and ai-driven framework for securing cloud containers," *IEEE Access*, 2025.

[7] B. L. Monroe, M. P. Colaresi, and K. M. Quinn, "Fightin' words: Lexical feature selection and evaluation for identifying the content of political conflict," *Political Analysis*, vol. 16, no. 4, pp. 372–403, 2008.